SOC Simulation Project

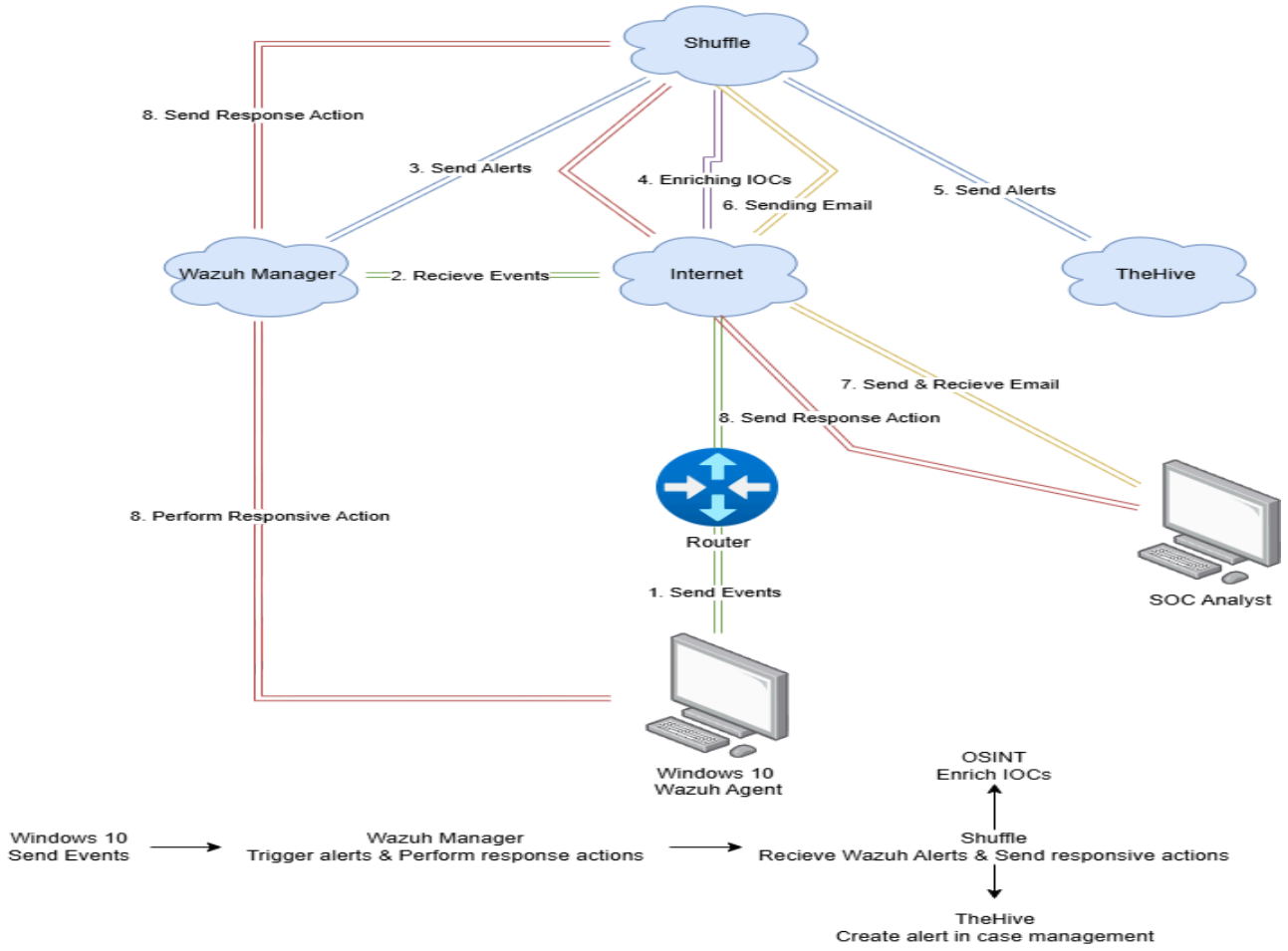
Summary: This project demonstrates the implementation of a cloud-based Security Operations Center (SOC) environment using an integrated stack of open-source security tools. The goal was to simulate a real-world enterprise environment where endpoint events are monitored, ingested, enriched, and automatically escalated for incident management.

🏗️ Lab Architecure

- **SIEM/XDR:** Wazuh Manager (Ubuntu)
- **SOAR:** Shuffle (Docker/Cloud)
- **Case Management:** TheHive (Ubuntu)
- **Endpoint:** Windows 11 (Wazuh Agent)
- **Automation:** OSINT Enrichment via VirusTotal/AbuseIPDB
- **Network:** DigitalOcean VPC with Cloud Firewall

🚀 Execution Phases

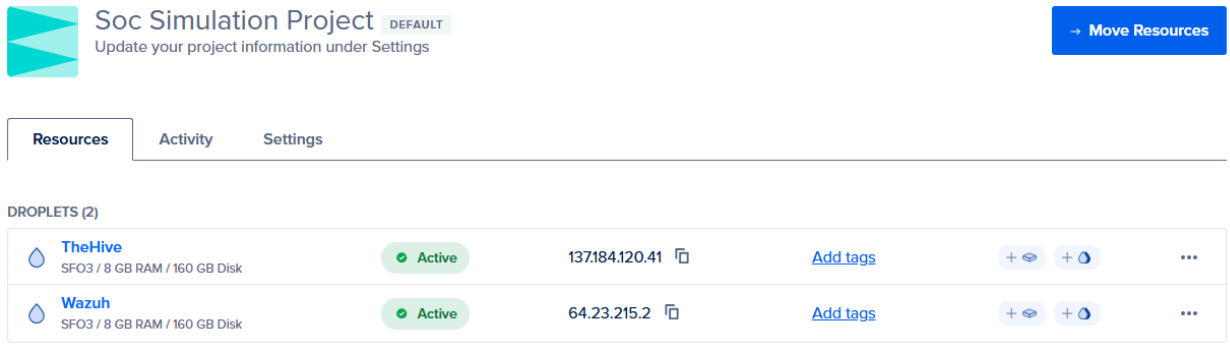
- **Phase 1**: Setup & Configuration
- **Phase 2**: Attack Simulation & Alerts & OSINT Enrichment (Shuffle SOAR Configured)



Phase 1: Setup & Configuration

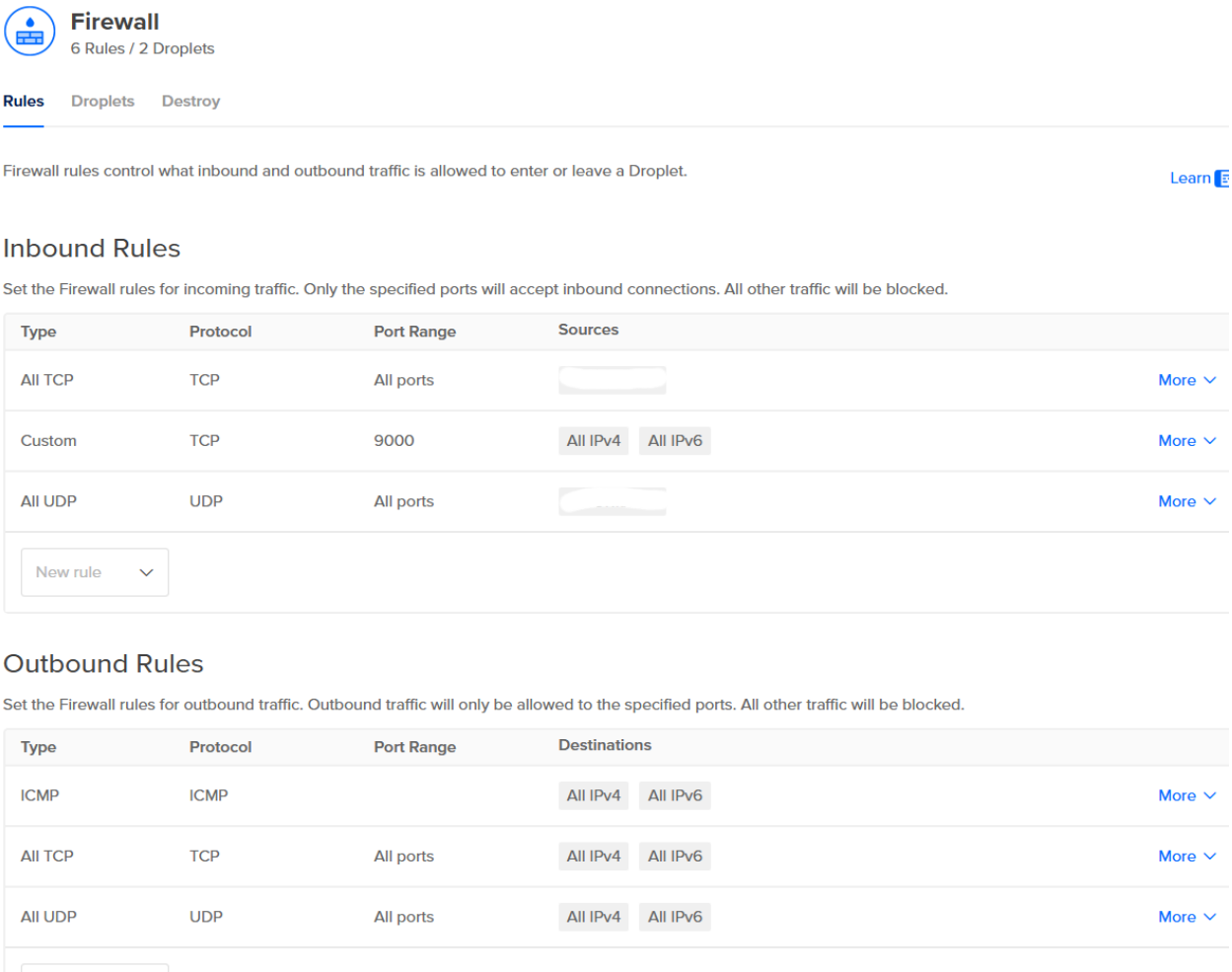
#1

Created two servers on DigitalOcean to run TheHive (Case Management) and Wazuh (SIEM Tool) as the manager for all events that will be managed by the Windows device.



#2

Configured a firewall for the two servers only allowing traffic from my IP address on protocol types TCP/UDP. I also configured a custom profile and set the port range to 9000 which will be used for TheHive Web interface/Dashboard



#3

Opened the Wazuh server console via PuTTy and installed Wazuh Manager/Dashboard for our Web Interface

```
02/03/2026 22:16:08 INFO: Starting Wazuh dashboard installation.
02/03/2026 22:16:52 INFO: Wazuh dashboard installation finished.
02/03/2026 22:16:52 INFO: Wazuh dashboard post-install configuration finished.
02/03/2026 22:16:52 INFO: Starting service wazuh-dashboard.
02/03/2026 22:16:53 INFO: wazuh-dashboard service started.
02/03/2026 22:17:24 INFO: Initializing Wazuh dashboard web application.
02/03/2026 22:17:25 INFO: Wazuh dashboard web application initialized.
02/03/2026 22:17:25 INFO: --- Summary ---
02/03/2026 22:17:25 INFO: You can access the web interface https://<wazuh-dashboard-ip>:443
    User: admin
    Password: [redacted]
02/03/2026 22:17:25 INFO: Installation finished.
root@Wazuh:~#
```

#4

Next I opened TheHive server console via PuTTy and installed TheHive Web interface/Dashboard, ElasticSearch as the Wazuh Indexer and used for storage, and Cassandra which will hold all of our data safely. This shows all services running and enabled for use.

```
root@TheHive: ~

● thehive.service - Scalable, Open Source and Free Security Incident Response Solutions
     Loaded: loaded (/usr/lib/systemd/system/thehive.service; enabled; preset: enabled)
     Active: active (running) since Tue 2026-03-10 03:43:25 UTC; 6 days ago
       Docs: https://strangebee.com
   Main PID: 28122 (java)
      Tasks: 122 (limit: 9487)
     Memory: 1.2G (peak: 1.2G)
        CPU: 6h 24min 3.230s
     CGroup: /system.slice/thehive.service
             └─28122 java -Dfile.encoding=UTF-8 -Dconfig.file=/etc/thehive/application.conf -Dlogger.file=/etc/thehive/logback.xml -Dpidfile.path=/dev/null @thehiv>

Mar 10 03:43:25 TheHive systemd[1]: Started thehive.service - Scalable, Open Source and Free Security Incident Response Solutions.

● cassandra.service - LSB: distributed storage system for structured data
     Loaded: loaded (/etc/init.d/cassandra; generated)
     Active: active (running) since Mon 2026-03-09 22:55:28 UTC; 6 days ago
       Docs: man:systemd-sysv-generator(8)
      Tasks: 87 (limit: 9487)
     Memory: 2.5G (peak: 2.5G)
        CPU: 3h 10min 38.326s
     CGroup: /system.slice/cassandra.service
             └─20533 /usr/bin/java -ea -da:net.openhft... -XX:+UseThreadPriorities -XX:+HeapDumpOnOutOfMemoryError -Xss256k -XX:+AlwaysPreTouch -XX:-UseBiasedLock>

Mar 09 22:55:28 TheHive systemd[1]: Starting cassandra.service - LSB: distributed storage system for structured data...
Mar 09 22:55:28 TheHive systemd[1]: Started cassandra.service - LSB: distributed storage system for structured data.

● elasticsearch.service - Elasticsearch
     Loaded: loaded (/usr/lib/systemd/system/elasticsearch.service; enabled; preset: enabled)
     Active: active (running) since Mon 2026-03-09 23:47:50 UTC; 6 days ago
       Docs: https://www.elastic.co
   Main PID: 23773 (java)
      Tasks: 83 (limit: 9487)
     Memory: 1.6G (peak: 1.6G)
        CPU: 38min 54.571s
     CGroup: /system.slice/elasticsearch.service
             ├─23773 /usr/share/elasticsearch/jdk/bin/java -Xms4m -Xmx64m -XX:+UseSerialGC -Dcli.name=server -Dcli.script=/usr/share/elasticsearch/bin/elasticsearc>
             ├─23832 /usr/share/elasticsearch/jdk/bin/java -Des.networkaddress.cache.ttl=60 -Des.networkaddress.cache.negative.ttl=10 -XX:+AlwaysPreTouch -Xss1m -D>
             └─23860 /usr/share/elasticsearch/modules/x-pack-ml/platform/linux-x86_64/bin/controller

Mar 09 23:47:19 TheHive systemd[1]: Starting elasticsearch.service - Elasticsearch...
```

#5

To make sure the services installed are using the right details such as IP addresses I went ahead into /etc/cassandra/cassandra.yaml (**cassandra acts as a database**) to configure some settings to make sure the listening address which is going to listen to TheHive's IP address and rpc address which will read and write data.

```
root@TheHive: ~
GNU nano 7.2                                    /etc/cassandra/cassandra.yaml

# Address or interface to bind to and tell other Cassandra nodes to connect to.
# You _must_ change this if you want multiple nodes to be able to communicate!
#
# Set listen_address OR listen_interface, not both.
#
# Leaving it blank leaves it up to InetAddress.getLocalHost(). This
# will always do the Right Thing _if_ the node is properly configured
# (hostname, name resolution, etc), and the Right Thing is to use the
# address associated with the hostname (it might not be). If unresolvable
# it will fall back to InetAddress.getLoopbackAddress(), which is wrong for production systems.
#
# Setting listen_address to 0.0.0.0 is always wrong.
#
listen_address: 137.184.120.41

# Set listen_address OR listen_interface, not both. Interfaces must correspond
# to a single address, IP aliasing is not supported.
# listen_interface: eth0
```

```
root@TheHive: ~
GNU nano 7.2                                    /etc/cassandra/cassandra.yaml
# native_transport_rate_limiting_enabled: false
# native_transport_max_requests_per_second: 1000000

# The address or interface to bind the native transport server to.
#
# Set rpc_address OR rpc_interface, not both.
#
# Leaving rpc_address blank has the same effect as on listen_address
# (i.e. it will be based on the configured hostname of the node).
#
# Note that unlike listen_address, you can specify 0.0.0.0, but you must also
# set broadcast_rpc_address to a value other than 0.0.0.0.
#
# For security reasons, you should not expose this port to the internet.  Firewall it if needed.
rpc_address: 137.184.120.41

# Set rpc_address OR rpc_interface, not both. Interfaces must correspond
```

#6

Next is configuring ElasticSearch so it binds to my Servers IP Address and the way to make ElasticSearch communicate with other services like Wazuh and TheHive is by opening its port 9200, For node.name: node-1, This acts as an instance so it's easier to track logs and monitoring tools.

```
root@TheHive: ~
  GNU nano 7.2
# ========================== Elasticsearch Configuration ==========================
cluster.name: NavsSim
node.name: node-1

path.data: /var/lib/elasticsearch
path.logs: /var/log/elasticsearch

# ---------------------------------- Network -----------------------------------
# Listen on your public IP and all interfaces
network.host: 137.184.120.41
http.host: 0.0.0.0
http.port: 9200

# --------------------------------- Discovery ----------------------------------
# This is the "Magic Fix" for single-node servers
discovery.type: single-node

# ------------------------ SECURITY CONFIGURATION ------------------------
```

#7

For TheHive I need to change its ownership from root to thehive, this is needed so no permission denied occurs when trying to run the service. The command ran for this step is cd /opt/thp, chown -R thehive:thehive /opt/thp

```
root@TheHive: /opt/thp
root@TheHive:/opt/thp# ls
thehive
root@TheHive:/opt/thp# ls -la
total 12
drwxr-xr-x 3 root root 4096 Mar  9 22:53 .
drwxr-xr-x 5 root root 4096 Mar  9 22:53 ..
drwxr-xr-x 5 root root 4096 Mar  9 22:53 thehive
root@TheHive:/opt/thp# chown -R thehive:thehive /opt/thp
root@TheHive:/opt/thp#
```

#8

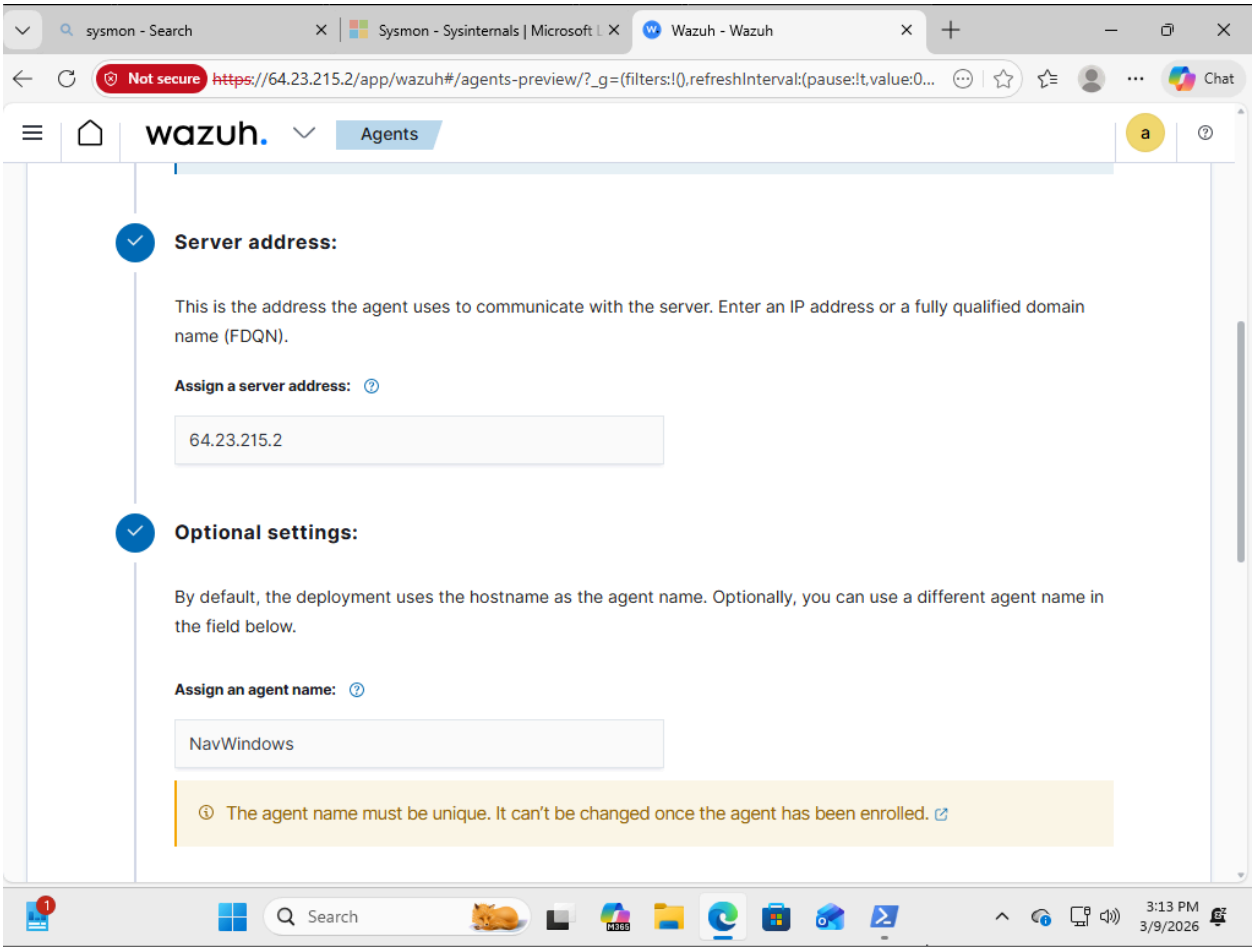
For this step I used the command nano /etc/thehive/application.conf the two things I changed here were the hostname which was my servers IP Address which were located under storage and index.search. The purpose of changing the hostname is so thehive knows how to communicate with cassandra and elasticsearch, it tells us where everything is located (**for this project all services besides Wazuh are ran on one server**). For application.baseUrl, this is used to generate links like email notifications or API calls.

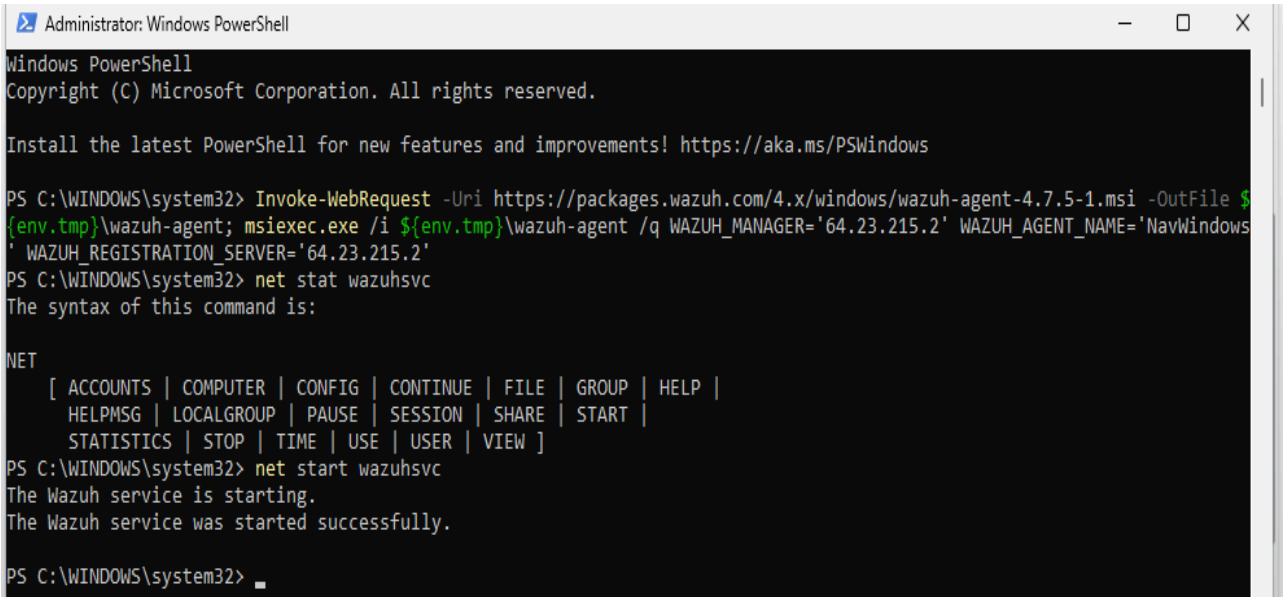
```
GNU nano 7.2                                    /etc/thehive/application.conf
# local Elasticsearch services without authentication.
db.janusgraph {
  storage {
    backend = cql
    hostname = ["137.184.120.41"]
    # Cassandra authentication (if configured)
    # username = "thehive"
    # password = "password"
    cql {
      cluster-name = NavsSim
      keyspace = thehive
    }
  }
  index.search {
    backend = elasticsearch
    hostname = ["137.184.120.41"]
    index-name = thehive
  }
}
```

```
# Service configuration
application.baseUrl = "
application.baseUrl = "http://137.184.120.41:9000"
play.http.context = "/"
```

#9

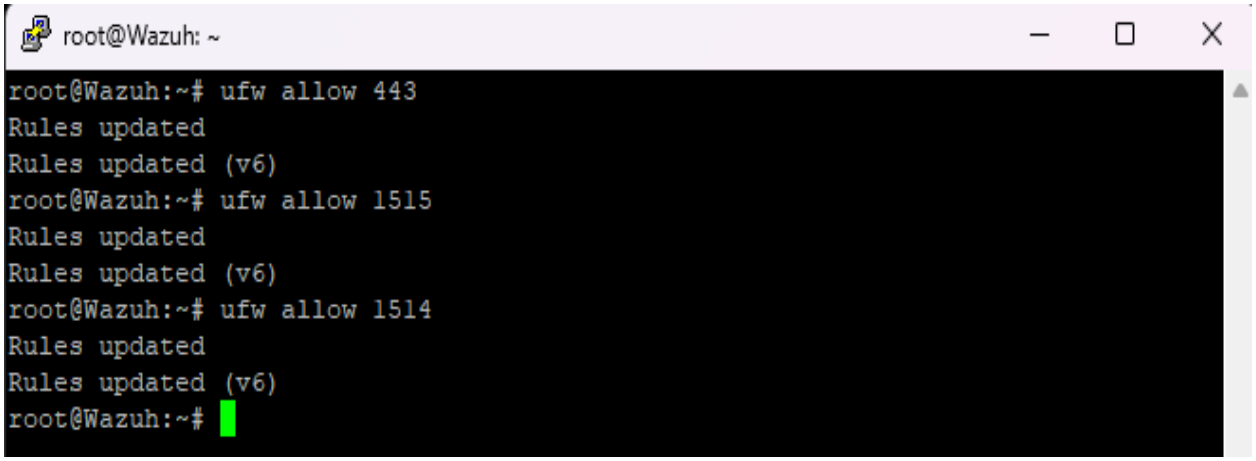
Now that the servers are both setup and configured properly I will be installing Sysmon onto my Windows 11 machine and deploying an agent for Wazuh so the machine can send information to the dashboard.





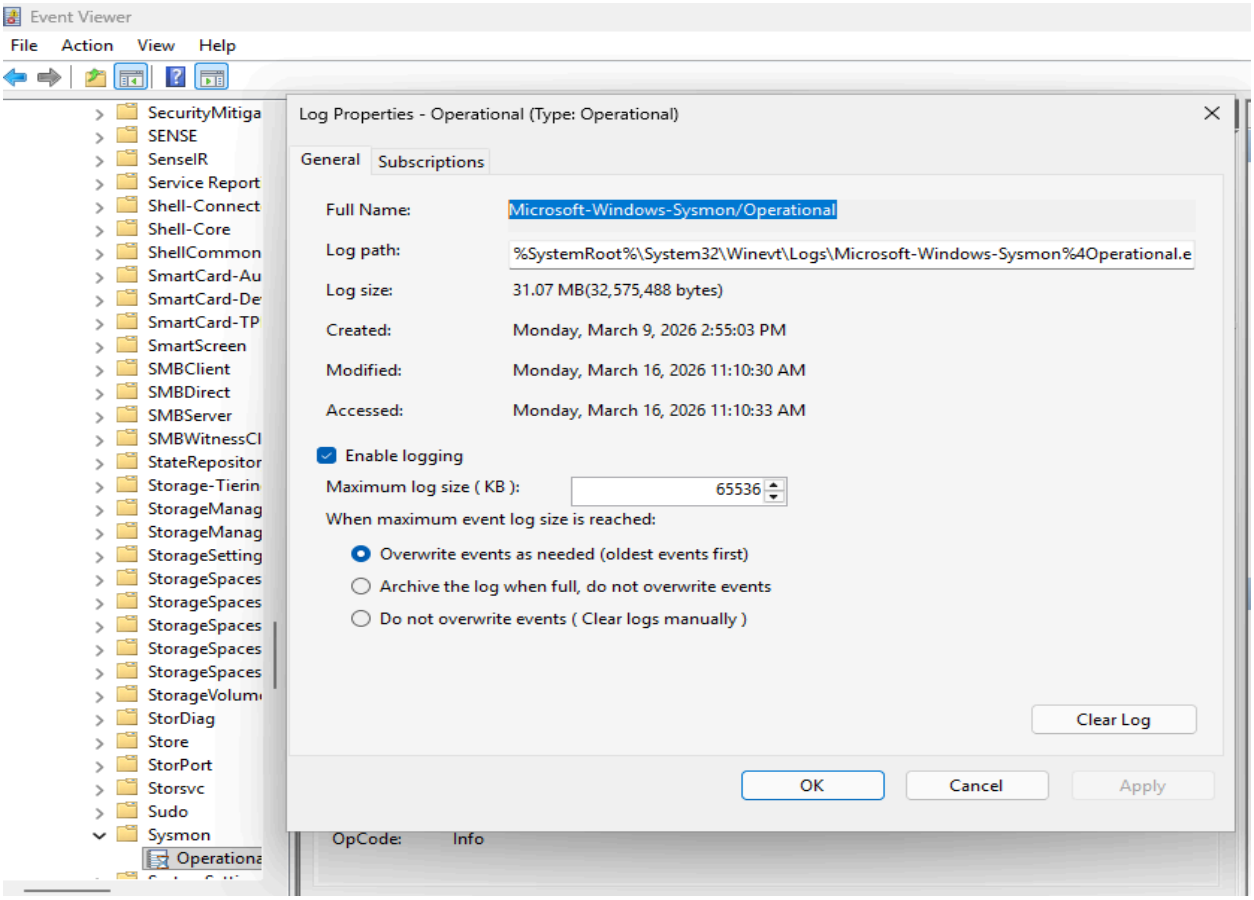
#10

After the agent was successfully running I went to the Wazuh server console and made sure the right ports were allowed (443, 1515, 1514) Port 1514 is used for sending all logs to the Manager. Port 1515 is used for registration, this means each log is given an ID and a security key before being sent to the Wazuh Manager, Port 443 is used for secure web browsing.



#11

Going back onto the windows machine I navigated to the ossec.conf file which appears after the Wazuh agent is installed, This shows where logs are being gathered from which is the Event Viewer, I changed the log analysis localfile to Microsoft-Windows-Sysmon/Operational so it gathers all information from Sysmon



Untitled | ossec.conf

File Edit View

```
    <queue_size>5000</queue_size>
    <events_per_second>500</events_per_second>
  </client_buffer>

  <!-- Log analysis -->
  <localfile>
    <location>Microsoft-Windows-Sysmon/Operational</location>
    <log_format>eventchannel</log_format>
  </localfile>

  <localfile>
    <location>active-response\active-responses.log</location>
    <log_format>syslog</log_format>
  </localfile>
```

#12

Logged into Wazuh Web Interface to see if the agent was successfully connected and to see if when I query for “sysmon” does it give related events.

STATUS

Active (1)
Disconnected (0)
Pending (0)
Never connected (0)

DETAILS

Active	Disconnected	Pending	Never connected
1	0	0	0

Last registered agent: NavWindows

Most active agent: NavWindows

Agents (1)

id!=000 and | Search

ID	Name	IP address	Group(s)	Operating system
001	NavWindows	192.168.174.128	default	Microsoft Windows 11 Pro 10.0.26200.7840

decoder.name	windows_eventchannel
id	1773118347.2149023
input.type	log
location	EventChannel
manager.name	Wazuh
rule.description	Powershell process created an executable file
rule.firedtimes	3
rule.groups	sysmon, sysmon_eid11_detections, windows
rule.id	92205
rule.level	9
rule.mail	false
rule.mitre.id	T1105
rule.mitre.tactic	Command and Control
rule.mitre.technique	Ingress Tool Transfer
timestamp	Mar 9, 2026 @ 21:52:27.887

Phase 2: Attack Simulation & Alerts & OSINT Enrichment (Shuffle SOAR Configured)

#13

For this step I simulated an attack using mimikatz which is used to steal passwords, hashes, pins, etc.. The main goal of this attack is to see if the logs pick it up and send it to the wazuh manager which it does.

```
Windows PowerShell
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Install the latest PowerShell for new features and improvements! https://aka
.ms/PSWindows

PS C:\Users\Targe\Downloads\mimikatz_trunk\x64> .\mimikatz.exe

  .#####.   mimikatz 2.2.0 (x64) #19041 Sep 19 2022 17:44:08
 .## ^ ##.  "A La Vie, A L'Amour" - (oe.eo)
 ## / \ ##  /*** Benjamin DELPY `gentilkiwi` ( benjamin@gentilkiwi.com )
 ## \ / ##       > https://blog.gentilkiwi.com/mimikatz
 '## v ##'       Vincent LE TOUX             ( vincent.letoux@gmail.com )
  '#####'        > https://pingcastle.com / https://mysmartlogon.com ***/

mimikatz #
PS C:\Users\Targe\Downloads\mimikatz_trunk\x64>
```

t _index	wazuh-alerts-4.x-2026.03.16
t agent.id	001
t agent.ip	192.168.174.128
t agent.name	NavWindows
t data.win.eventdata.commandLine	\"C:\\Users\\Targe\\Downloads\\mimikatz_trunk\\x64\\mimikatz.exe\"
t data.win.eventdata.company	gentilkiwi (Benjamin DELPY)
t data.win.eventdata.currentDirectory	C:\\Users\\Targe\\Downloads\\mimikatz_trunk\\x64\\

#14

To track an attack like mimikatz we have to configure the ossec.conf file located on the wazuh server console and change logall and logall_json to yes, this allows all events to be captured not just the ones triggered by event IDs. Another crucial step that will actually pick up the logs and deliver them to the indexer is filebeat, enabling archives > true will help with that.

```
root@Wazuh: ~
GNU nano 6.2                    /var/ossec/etc/ossec.conf
<!--
 Wazuh - Manager - Default configuration for ubuntu 22.04
 More info at: https://documentation.wazuh.com
 Mailing list: https://groups.google.com/forum/#!forum/wazuh
-->

<ossec_config>
  <global>
    <jsonout_output>yes</jsonout_output>
    <alerts_log>yes</alerts_log>
    <logall>yes</logall>
    <logall_json>yes</logall_json>
```

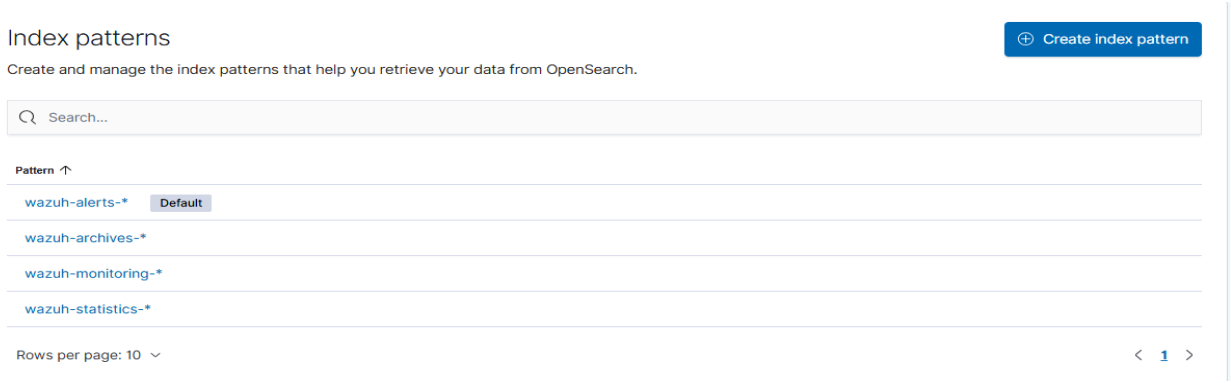
```
root@Wazuh: /var/ossec/logs/archives
GNU nano 6.2                    /etc/filebeat/filebeat.yml
    - /etc/filebeat/certs/root-ca.pem
  ssl.certificate: "/etc/filebeat/certs/wazuh-server.pem"
  ssl.key: "/etc/filebeat/certs/wazuh-server-key.pem"
setup.template.json.enabled: true
setup.template.json.path: '/etc/filebeat/wazuh-template.json'
setup.template.json.name: 'wazuh'
setup.ilm.overwrite: true
setup.ilm.enabled: false

filebeat.modules:
  - module: wazuh
    alerts:
      enabled: true
    archives:
      enabled: true

logging.level: info
logging.to_files: true
logging.files:
  path: /var/log/filebeat
```

#15

The part that will be picking up the logs and delivering it to a certain index pattern will be named wazuh-archives-* that I created after configuring the filebeat.yml file. To create it navigate to **Management > Stack Management > Index Patterns**



#16

To go more in depth I created a rule id so whenever mimikatz is detected on the machine it will create a custom message along with the severity level

```
root@Wazuh: ~
GNU nano 6.2          /var/ossec/etc/rules/local_rules.xml *
<!-- Example -->
<group name="local,syslog,sshd,">

  <!--
  Dec 10 01:02:02 host sshd[1234]: Failed none for root from 1.1.1.1 port 1066
  -->
  <rule id="100001" level="5">
    <if_sid>5716</if_sid>
    <srcip>1.1.1.1</srcip>
    <description>sshd: authentication failed from IP 1.1.1.1.</description>
    <group>authentication_failed,pci_dss_10.2.4,pci_dss_10.2.5,</group>
  </rule>

  <rule id="100002" level="15">
    <if_group>sysmon_event1</if_group>
    <field name="win.eventdata.originalFileName" type="pcre2">(?i)mimikatz\.exe
    <description>Mimikatz Usage is being detected</description>
    <mitre>
        <id>T1003M</id>
    </mitre>

^G Help      ^O Write Out ^W Where Is  ^K Cut       ^T Execute   ^C Location
^X Exit      ^R Read File ^\ Replace   ^U Paste     ^J Justify   ^/ Go To Line
```

manager.name: Wazuh + Add filter

wazuh-alerts-*

Search field nam

Filter by type 0

Selected fields

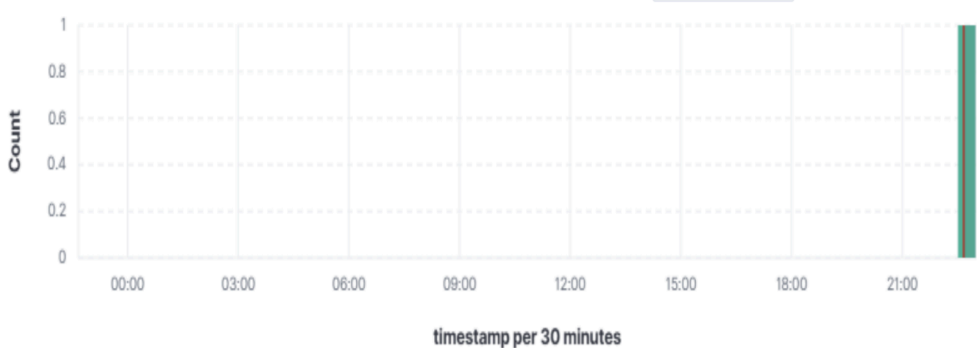
- t agent.name
- t rule.description
- t rule.id
- # rule.level

Available fields

- t agent.id
- t agent.ip
- t data.arch
- t data.dpkg_status

1 hit

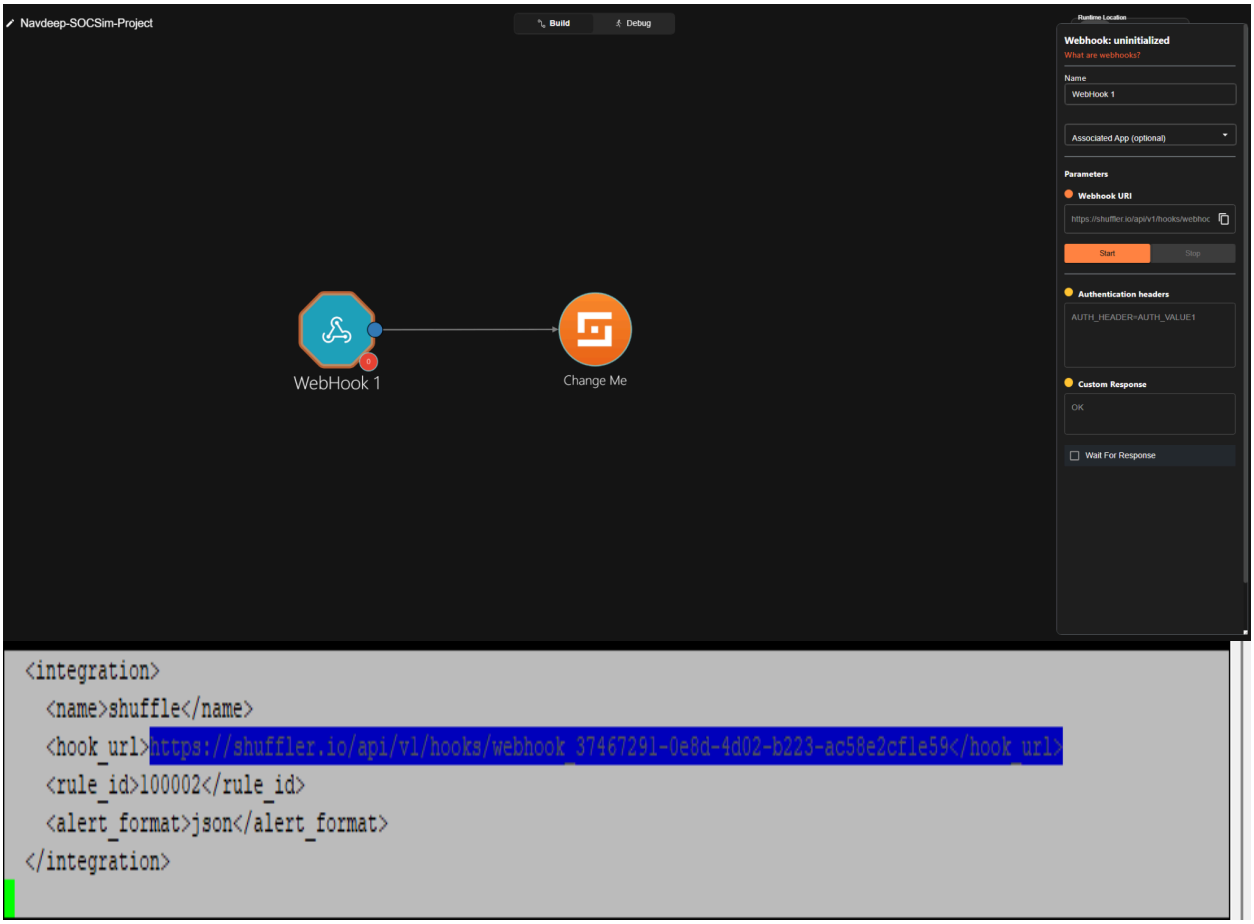
Mar 8, 2026 @ 22:40:07.202 - Mar 9, 2026 @ 22:40:07.202 Auto



Time	agent.name	rule.description	rule.level	rule.id
Mar 9, 2026 @ 22:34:15.318	NavWindows	Mimikatz Usage is being detected	15	100002

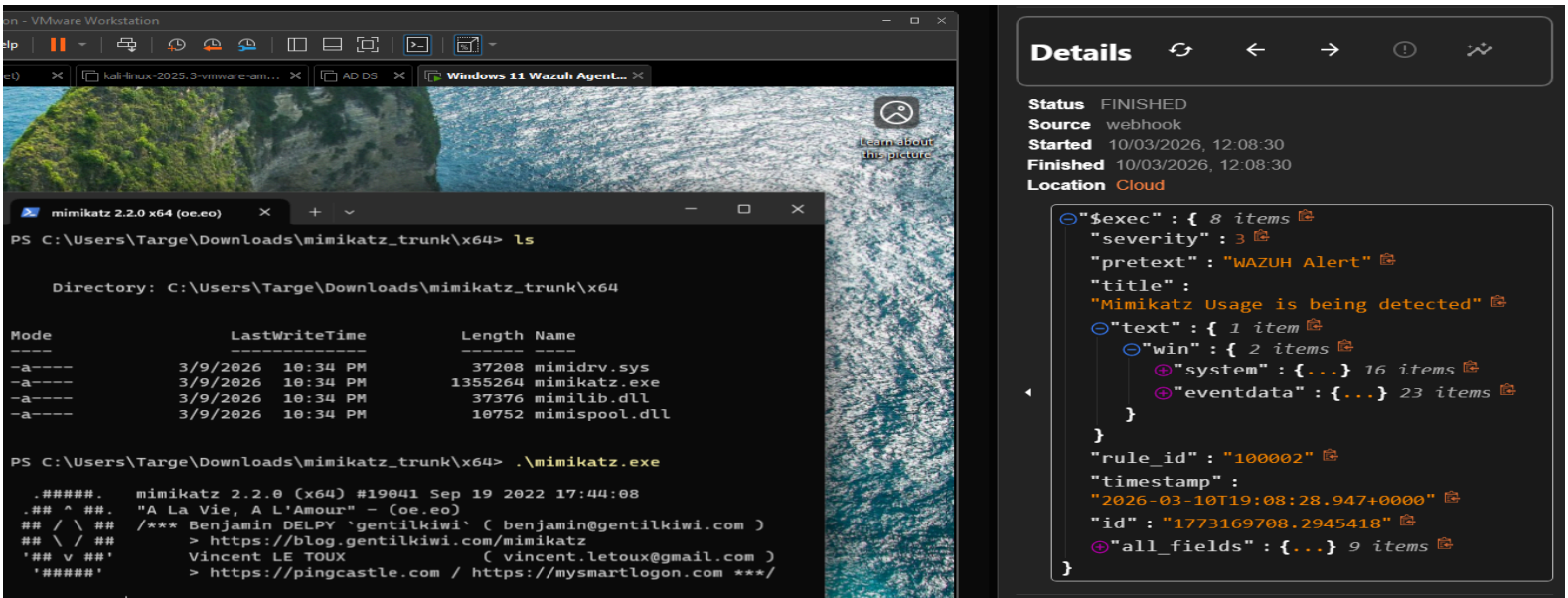
#17

Now I will be implementing SOAR using Shuffle to automate the process of an attack being detected. I first connected a webhook on shuffler.io and used that URI in the ossec.conf file so when a certain criteria matches like a severity level it will send a POST to that webhook for further automation.



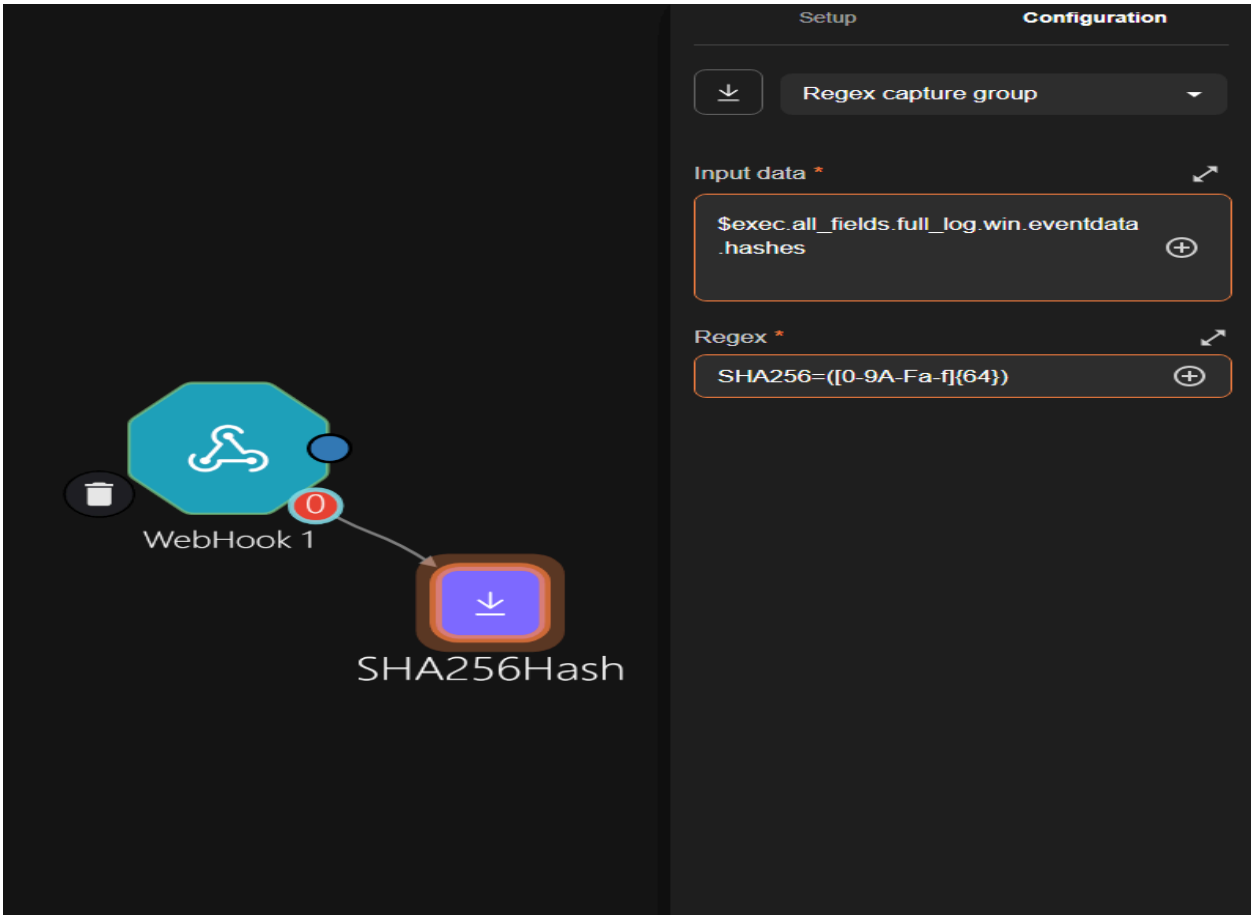
#18

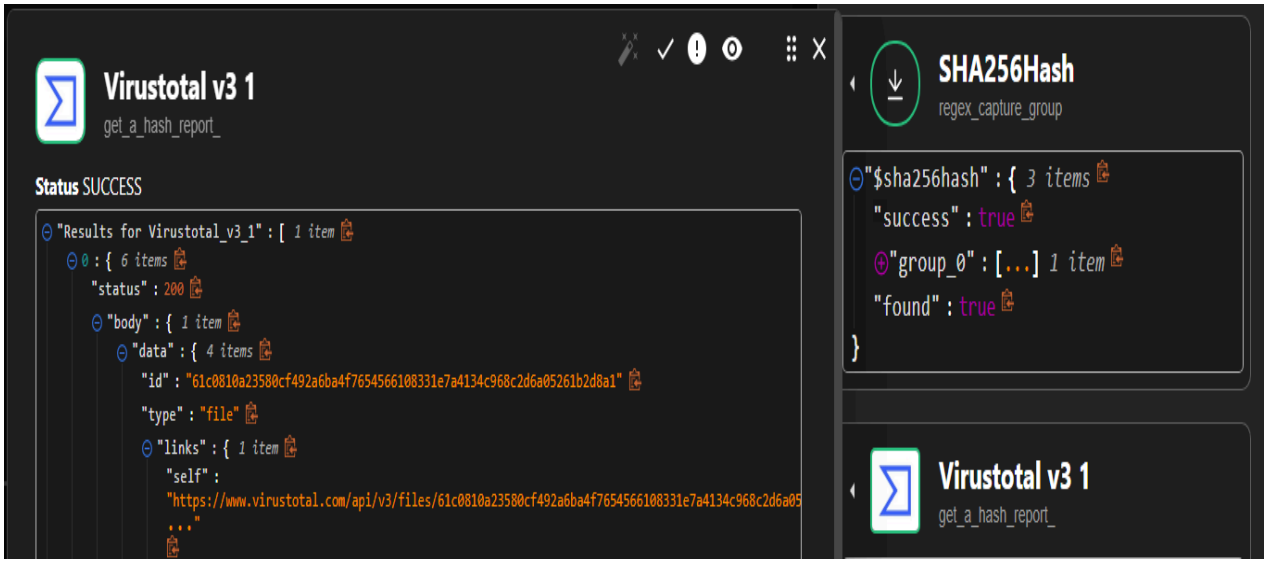
Now when I run another mimikatz attack it will gather that event and send it out to the webhook to be displayed



#19

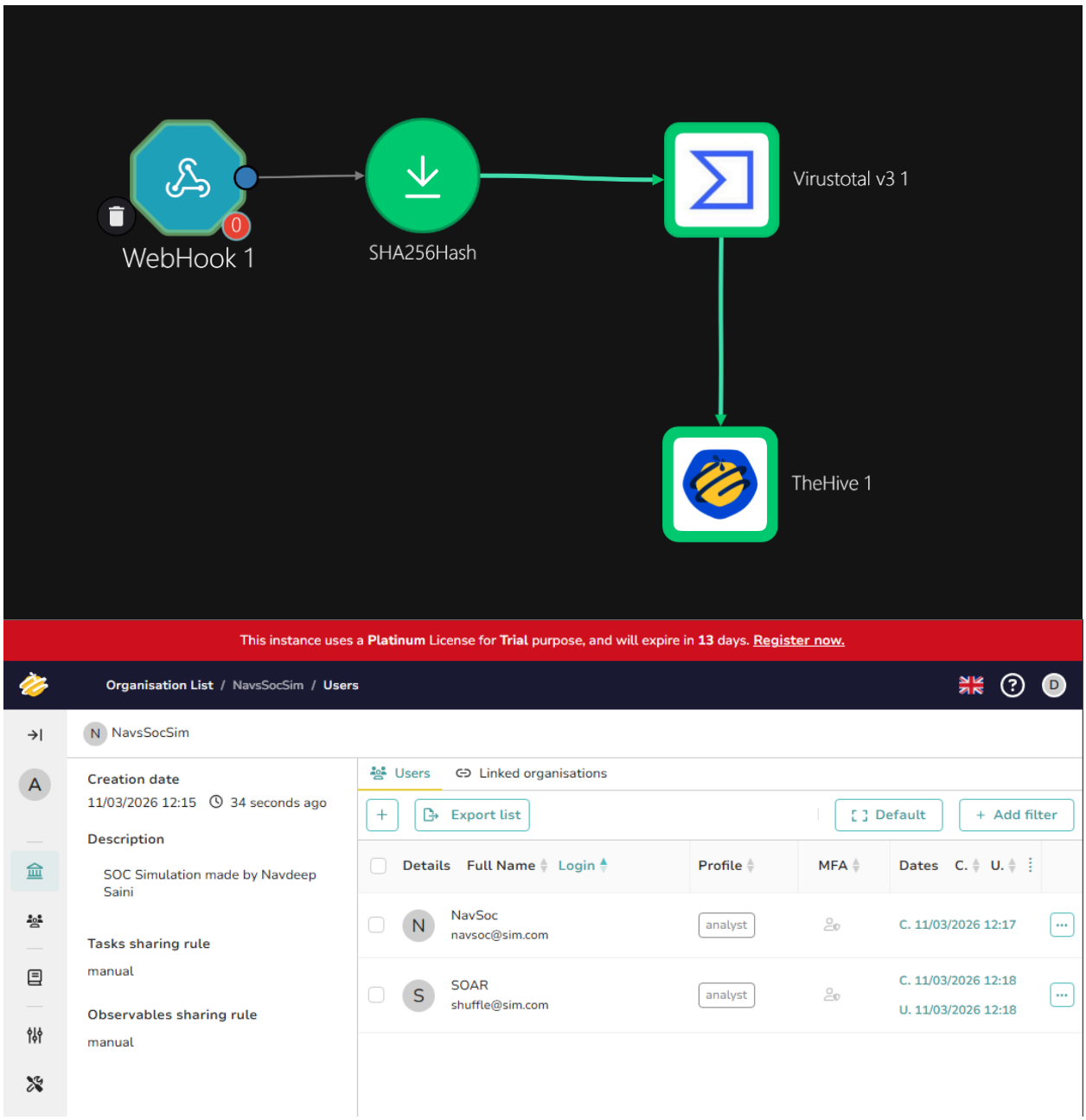
I will add a SHA256 which is a hashing algorithm using 256-bit value. The input data shows which field to target and the regex is what isolates the hash from the Sysmon event. This overall ensures that the system only picks up on high indicators. For the hash to be valuable I connected it with VirusTotal to detect what the hash is whether it's a file and if it’s secure, VirusTotal shows SUCCESS meaning it was able to check the hash it was given and return a status 200 with the hash ID.

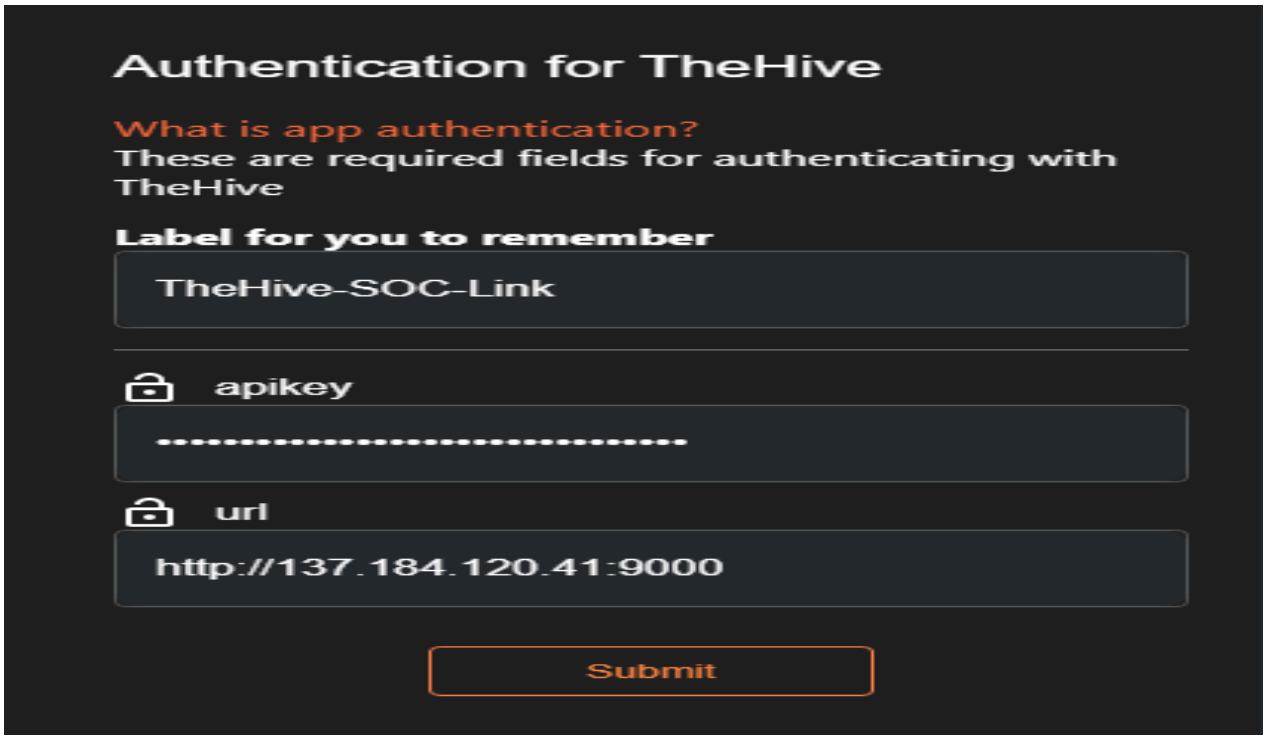




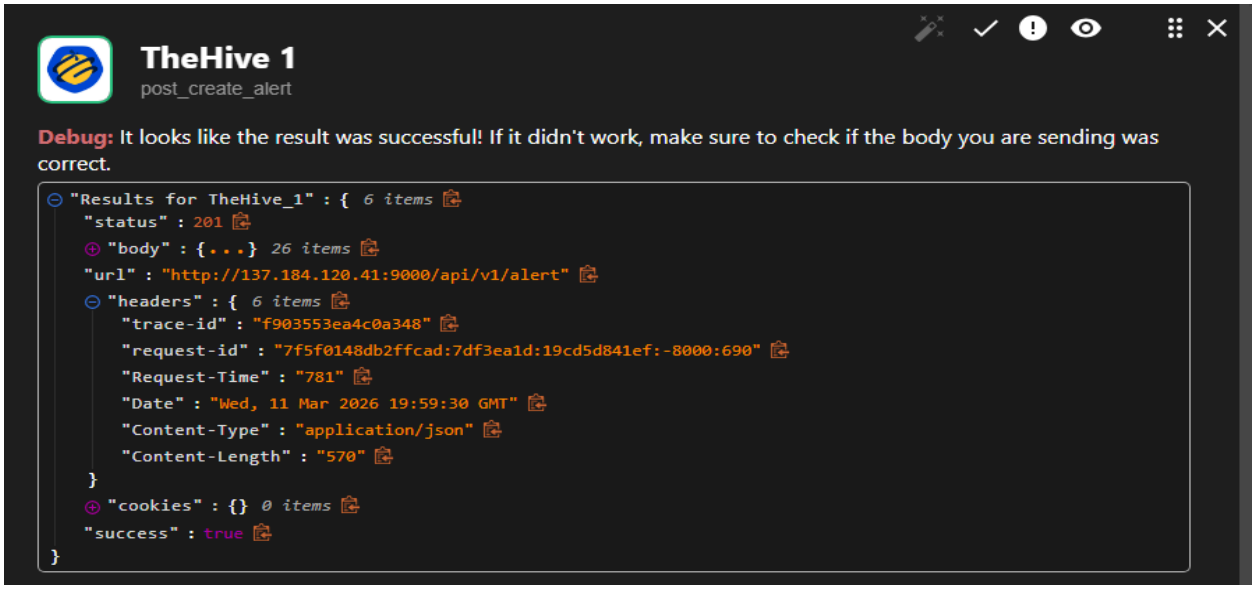
#20

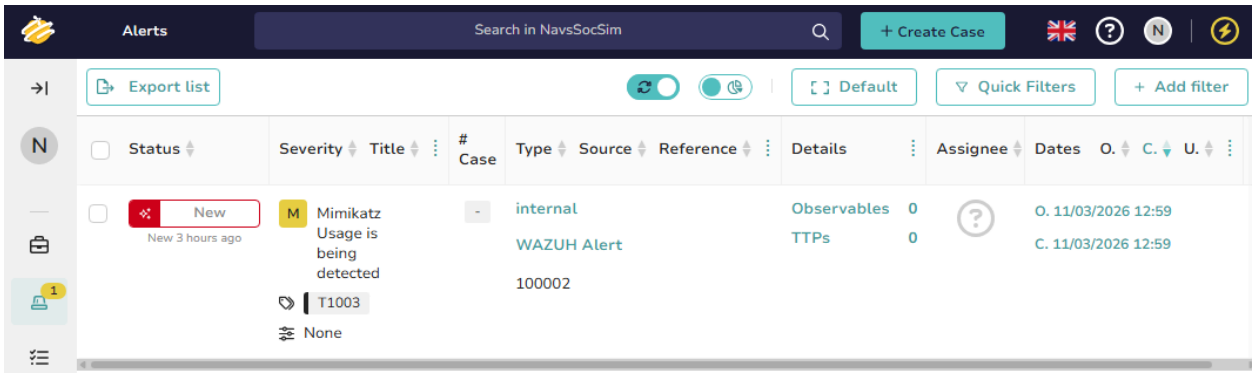
I then connected TheHive to the VirusTotal application to send alerts to the SOC analyst, first I created an API key with the IP address of TheHive's Website (http://137.184.120.41:9000). I then created an organization within TheHive's Interface and created Two profiles NavSoc which is the analyst and SOAR which will be used as the service since we are connecting shuffle with an API key





#21
Lastly I ran the test to see if it works and it successfully went through VirusTotal to TheHive and generated an alert which was then sent to the SOC Analyst (NavSoc) for further escalation.




Alerts
Search in NavsSocSim
+ Create Case
Export list
Default
Quick Filters
+ Add filter
Status
Severity
Title
Case
Type
Source
Reference
Details
Assignee
Dates
O.
C.
U.
New
New 3 hours ago
M
Mimikatz Usage is being detected
T1003
None
-
internal
WAZUH Alert
100002
Observables 0
TTPs 0
O. 11/03/2026 12:59
C. 11/03/2026 12:59